



การใช้ REFS และ STATE

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

การสร้างค่าอ้างอิง (Refs)

ตามปกติแล้ว วิธีการอ้างอิงถึงอิลิเมนต์ในแบบของจาวาสคริปต์ จะมีเมธอดให้เลือกใช้หลายอัน เช่น getElementById(), getElementsByName(), getElementsByTagName(), getElementByClassName(), querySelector() เป็นต้น แต่เนื่องจากการสร้างเว็บเพจตามหลักการของ React จะเป็นการนำหลายๆ คอมโพเนนต์มารวมกัน ซึ่งในแต่ละคอมโพเนนต์อาจมีองค์ประกอบบางอย่างที่มีลักษณะเหมือนกัน เช่น สมมติว่ามี <button id="bt"> ในหลายๆ คอมโพเนนต์ ถ้าเราอ้างอิงด้วย document.getElementById('bt') จะเกิดปัญหาว่า เราหมายถึง button ที่อยู่ในคอมโพเนนต์ไหน แม้การใช้งานจริงมันก็จะเลือกอันแรกที่เจอมาให้เรา แต่ผลลัพธ์ที่ได้อาจไม่ตรงตามเป้าหมายที่ต้องการ ดังนั้น เราไม่ควรใช้เมธอดของจาวาสคริปต์ดังที่กล่าวมาเพื่อเข้าถึงอิลิเมนต์ในคอมโพเนนต์ แต่ควรสร้างค่าอ้างอิงของอิลิเมนต์ (Refs หรือ Reference) แล้วเข้าถึงผ่านค่าอ้างอิงนั้นแทน ซึ่งมีหลักการบางอย่างที่แตกต่างกันระหว่าง Class Component และ Function Component ดังรายละเอียดต่อไปนี้

การสร้างค่าอ้างอิง (Refs)

- การสร้างค่าอ้างอิงใน Class Component

กรณีของ Class Component ให้สร้างค่าอ้างอิงด้วยเมธอด `createRef()` ซึ่งเราอาจนำเข้าเมธอดนี้โดยตรงจากโมดูล 'react' หรือจะเรียกใช้งานผ่านคอมโพเนนต์ React ก็ได้ เช่น

//วิธีที่ 1

```
import React, { createRef } from 'react'
```

```
class MyClass extends React.Component {  
  button = createRef() //หรือจะกำหนดไว้ใน constructor ก็ได้  
  ...  
}
```

//วิธีที่ 2

```
import React from 'react'
```

```
class MyClass extends React.Component {  
  button = React.createRef() //หรือจะกำหนดไว้ใน constructor ก็ได้  
  ...  
}
```

การสร้างค่าอ้างอิง (Refs)

ภายในคอมโพเนนต์ หลังจากที่เราสร้างตัวแปรสำหรับอ้างอิงถึงคอมโพเนนต์ด้วย `createRef()` ดังโค้ดที่ผ่านมา ต่อไปก็ต้องเชื่อมโยงกับอีลีเมนต์เป้าหมายโดยการนำตัวแปรดังกล่าวมากำหนดให้แก่พร็อพเพอร์ตี้ **ref** ของอีลีเมนต์นั้น เช่น

```
buttonOK = React.createRef()
...
return <button ref={this.buttonOK}>OK</button>
```

ถ้ามีอีลีเมนต์อื่นๆ อีก ที่เราต้องอ้างอิง ก็ให้สร้างค่าอ้างอิงด้วย `createRef()` แยกแต่ละอีลีเมนต์ในแบบเดียวกันไปจนครบ เช่น

```
div = createRef()
span = createRef()
button = createRef()
...
return (
  <div ref={this.div}>
    <span ref={this.span}>...</span>
    <button ref={this.button}>...</button>
  </div>
)
```

การสร้างค่าอ้างอิง (Refs)

เมื่อเราต้องการเข้าถึงอิลิเมนต์ที่ได้สร้างค่าอ้างอิงเอาไว้ เช่น เพื่ออ่านหรือกำหนดข้อมูล หรือการเปลี่ยนแปลง สไตล์ หรืออื่นๆ ให้อ้างอิงด้วยรูปแบบดังนี้

```
this.<ref>.current.<property_or_method>
```

- ref คือชื่ออ้างอิงที่เรากำหนดได้เอาไว้
- current ต้องระบุลงไปด้วย เพื่อบ่งชี้ว่าเป็นอิลิเมนต์ที่อยู่ในคอมโพเนนต์ปัจจุบัน ทั้งนี้เพราะในคอมโพเนนต์อื่นๆ ซึ่งนำมารวมกันเป็นเว็บเพจ อาจมีอิลิเมนต์ที่ระบุชื่ออ้างอิงซ้ำกันก็ได้
- property_or_method คือพร็อพเพอร์ตี้หรือเมธอดของอิลิเมนต์ในแบบ DOM ที่เราต้องการเข้าถึงเช่น innerHTML, innerText, value (อินพุตของฟอร์ม), style, focus(), submit() และอื่นๆ ทั้งหมดตามข้อกำหนดของจาวาสคริปต์ เช่น

```
const title = this.button.current.innerText  
this.span.current.innerHTML = '<b>Hello</b>'  
this.div.current.style.backgroundColor = 'yellow'
```

การสร้างค่าอ้างอิง (Refs)

- การสร้างค่าอ้างอิงใน Function Component

กรณีของ Function Component ให้สร้างค่าอ้างอิงด้วยเมธอด `useRef()` ซึ่งจัดอยู่ในกลุ่ม React Hook หรือ `React.createRef()` วิธีใดวิธีหนึ่ง ดังโค้ดถัดไป แล้วเชื่อมโยงกับอิลิเมนต์เป้าหมายเช่นเดียวกับ Class Component เพียงแต่ไม่ต้องระบุคำว่า `this` เพราะเป็น Function Component

//วิธีที่ 1

```
import React, { useRef } from 'react'

function MyFunction() {
  const div = useRef()
  const button = useRef()
  ...
  return (
    <div ref={div}>
      <button ref={button}>OK</button>
    </div>
  )
}
```

//วิธีที่ 2

```
import React from 'react'

function MyFunction() {
  const div = React.useRef() //หรือใช้ React.createRef()
  const button = React.useRef() //หรือใช้ React.createRef()
  ...
  return (
    <div ref={div}>
      <button ref={button}>OK</button>
    </div>
  )
}
```

การสร้างค่าอ้างอิง (Refs)

ส่วนการเข้าถึงอิลิเมนต์ที่สร้างค่าอ้างอิงเอาไว้ ก็ทำแบบเดียวกับ Class Component เพียงแต่ไม่ต้องระบุคำว่า `this` เช่น

```
const title = button.current.innerText  
div.current.style.backgroundColor = 'yellow'
```

สำหรับการใช้งานโดยรวม ให้ดูจากการทดสอบต่อไปนี้ โดยภายในคอมโพเนนต์จะประกอบด้วย `span` 2 อันที่กำหนดตัวเลขไว้อย่างแน่นอน และ `span` อีก 2 อันสำหรับแสดงเครื่องหมายที่ใช้คำนวณ และผลลัพธ์จากการคำนวณตามลำดับ นอกจากนี้ก็จะมีปุ่มสำหรับคลิกเลือกเครื่องหมาย เมื่อคลิกปุ่มใด จะอ่านค่าจาก `span` ที่แสดงตัวเลขและเครื่องหมายบนปุ่มนั้น แล้วนำไปคำนวณเมื่อได้ผลที่ออกมาก็นำไปแสดงใน `span` ผลลัพธ์

การสร้างค่าอ้างอิง (Refs)

react/app1/src/refs-class.js

```
import React, { createRef } from 'react'

export default class RefsClass extends React.Component {
  num1 = createRef()
  num2 = createRef()
  operator = React.createRef()
  result = React.createRef()

  calculate = (ev) => {
    let op = ev.target.innerText
    let n1 = parseInt(this.num1.current.innerText)
    let n2 = parseInt(this.num2.current.innerText)
    let r = eval(`${n1} ${op} ${n2}`)
    this.result.current.innerText = r
    this.operator.current.innerText = op
  }
}
```

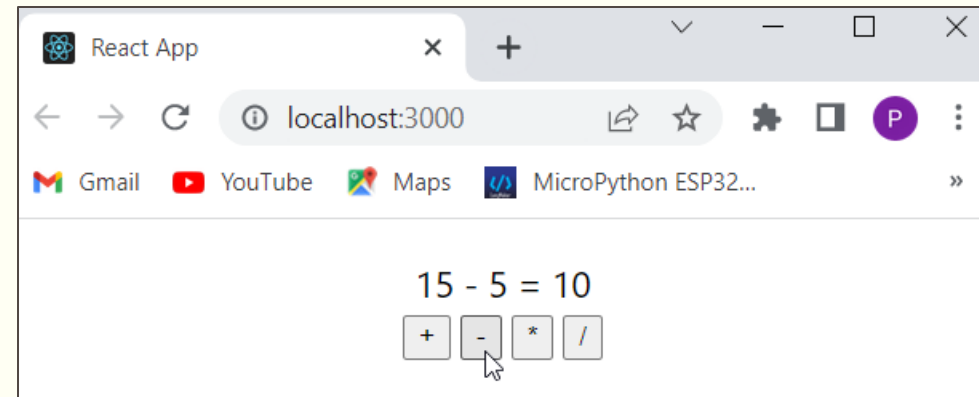
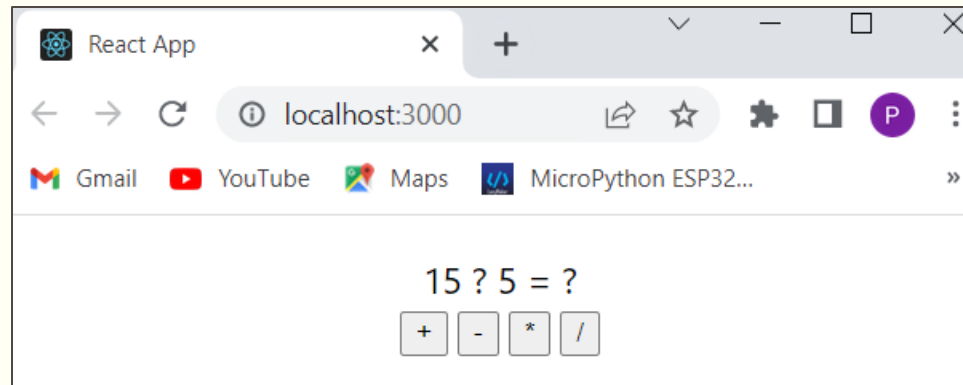
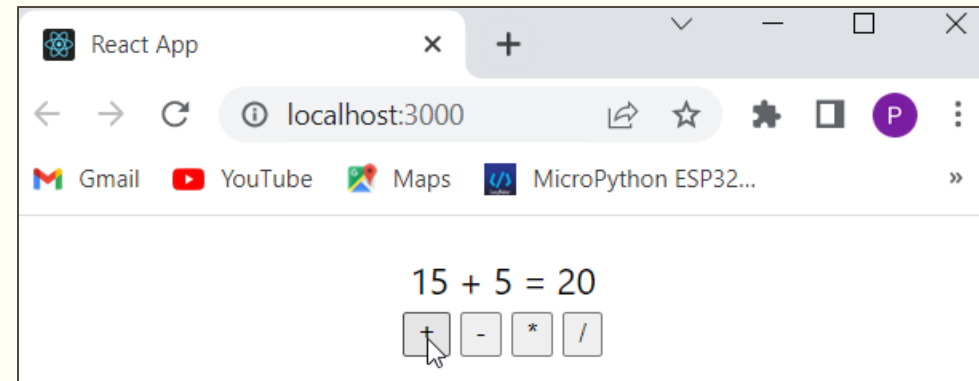
```
render() {
  return (
    <div style={{ textAlign: 'center', marginTop: 20, fontSize: 18 }}>
      <span ref={this.num1}>15</span>&nbsp;
      <span ref={this.operator}>?</span>&nbsp;
      <span ref={this.num2}>5</span>&nbsp;
      <span>=</span>&nbsp;
      <span ref={this.result}>?</span>
    </div>
    <button onClick={this.calculate}>+</button>&nbsp;
    <button onClick={this.calculate}>-</button>&nbsp;
    <button onClick={this.calculate}>*</button>&nbsp;
    <button onClick={this.calculate}>/</button>
  </div>
)
}
```


การสร้างค่าอ้างอิง (Refs)

react/app1/src/App.js

```
import React from 'react'
import RefsClass from './refs-class'

export default function App() {
  return <RefsClass/>
}
```



การสร้างค่าอ้างอิง (Refs)

react/app1/src/refs-func.js

```
import React, { ..... } from 'react'

export default function RefsFunc() {
  const num1 = .....
  const num2 = useRef()
  const operator = React.useRef()
  const result = .....

  const calculate = (.....) => {
    let op = ev.target.innerText
    let n1 = parseInt(num1.current.innerText)
    let n2 = parseInt(.....)
    let r = eval(`${.....} ${.....} ${n2}`)
    result.current.innerText = r
    operator.current.innerText = .....
  }
```

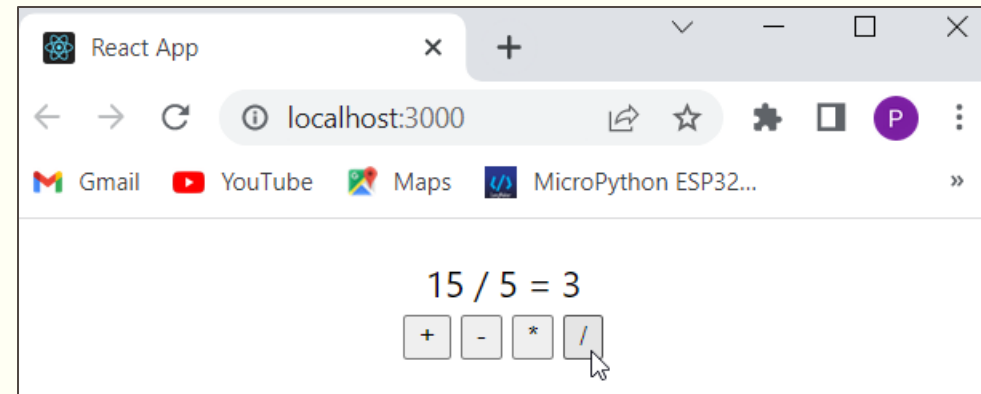
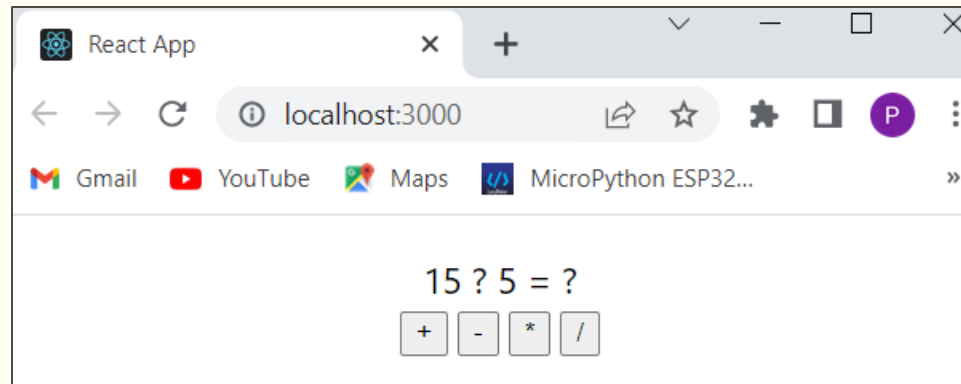
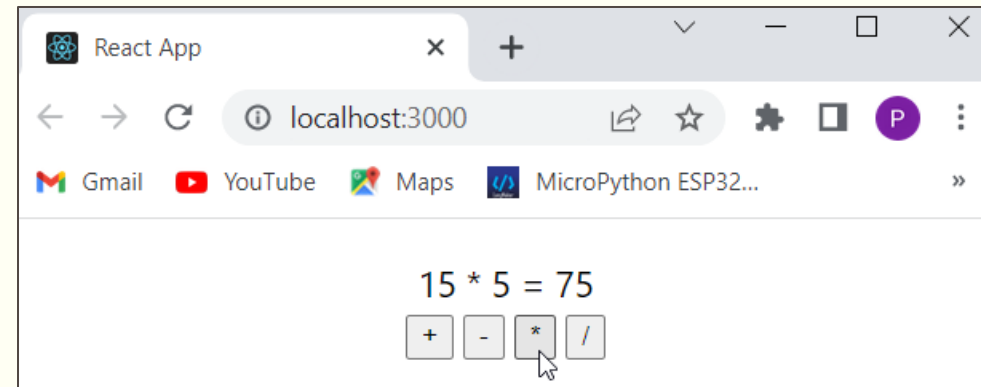
```
return (
  <div style={{ textAlign: 'center', marginTop: 20, fontSize: 18 }}>
    <span ref={num1}>.....</span>&nbsp;
    <span ref={operator}>.....</span>&nbsp;
    <span ref={.....}>5</span>&nbsp;
    <span>=</span>&nbsp;
    <span ref={.....}>?</span>
    <div>
      <button onClick={.....}>+</button>&nbsp;
      <button onClick={.....}>-</button>&nbsp;
      <button onClick={.....}>*</button>&nbsp;
      <button onClick={calculate}>.....</button>
    </div>
  </div>
)
```

```
}
```

การสร้างค่าอ้างอิง (Refs)

react/app1/src/App.js

```
import .....  
import ..... from './refs-func'  
  
export default function App() {  
  return .....  
}
```



การสร้างค่าอ้างอิง (Refs)

- การสร้างค่าอ้างอิงแบบอาร์เรย์

บางครั้ง อิลิเมนต์ที่เราต้องการอ้างอิงอาจมีเป็นจำนวนมาก เช่น เซลล์หรือแถวของตาราง จึงอาจยุ่งยากต่อการสร้างค่าอ้างอิงให้ครบทุกอิลิเมนต์ ซึ่งในลักษณะเช่นนี้ เราอาจเปลี่ยนมาใช้วิธีการแบบอาร์เรย์เพื่อสร้างค่าอ้างอิงหลายๆ อิลิเมนต์ในตัวแปรเดียวกัน ดังแนวทางต่อไปนี้

- กำหนดตัวแปรเพื่อให้เป็นค่าอ้างอิงสำหรับอิลิเมนต์ในกลุ่มนั้นเพียงตัวเดียว พร้อมระบุค่าเริ่มต้นให้เป็นอาร์เรย์ว่าง
- ในขั้นตอนการแสดงผลคอมโพเนนต์ ซึ่งเราต้องกำหนดค่าอ้างอิงของอิลิเมนต์ โดยวิธีที่สะดวกที่สุดก็คือใช้ Arrow Function พร้อมระบุเลขลำดับให้กับอิลิเมนต์นั้น เช่น

```
const buttons = React.useRef([])
...
return (
  <div>
    <button ref={el => buttons.current[0] = el}>One</button><br/>
    <button ref={el => buttons.current[1] = el}>Two</button><br/>
    <button ref={el => buttons.current[2] = el}>Three</button><br/>
    ...
  </div>
)
```

การสร้างค่าอ้างอิง (Refs)

- หากมีอีลีเมนต์ชนิดเดียวกันเป็นจำนวนมากที่กำหนดค่าอ้างอิง เพื่อลดความยุ่งยากในการเขียนโค้ดเราควรนำค่าบางอย่างของแต่ละอีลีเมนต์ มาสร้างเป็นอาร์เรย์ แล้วใช้เมธอด `map()` เพื่อแสดงอีลีเมนต์ เช่น จากโค้ดที่ผ่านมา อาจแก้ไขเป็นดังนี้

```
const buttons = React.useRef([])
const title = ['One', 'Two', 'Three', 'Four', 'Five', ...]
...
return (
  <div>
  {
    title.map((t,i) => {
      return (
        <button ref={el => buttons.current[i] = el}>{t}</button><br/>
      )
    })
  }
  </div>
)
```

การสร้างค่าอ้างอิง (Refs)

- หากเราต้องการอ้างอิงถึงอีลีเมนต์ที่สร้างค่าอ้างอิงแบบอาร์เรย์ จะต้องระบุเลขลำดับของอีลีเมนต์นั้นลงไปด้วย เช่น

```
let bt1Title = buttons.current[0].innerText  
buttons.current[1].style.color = 'red'
```

การสร้างค่าอ้างอิง (Refs)

ตัวอย่างถัดไปเป็นการแสดงตาราง โดยในแต่ละแถวจะมีปุ่มสำหรับคลิกเพื่อลบแถวนั้นออกไป ทั้งนี้แม้จะทำได้หลายวิธี แต่เพื่อให้สอดคล้องกับที่เราศึกษาในหัวข้อนี้ จะสร้างค่าอ้างอิงสำหรับแต่ละแถวของตารางในแบบอาร์เรย์ แล้วที่แต่ละปุ่มก็จะแนบลำดับแถวเพื่อส่งไปยังตัวจัดการอีเวนต์สำหรับใช้เป็นตัวกำหนดให้ลบแถวในลำดับนั้นออกจากตาราง ส่วนรายละเอียดปลีกย่อยอื่นๆ ให้ดูจากโค้ดต่อไปนี้

react/app1/src/refs-array.js

```
import React from 'react'
```

```
export default function ..... {
```

```
  const table = React.useRef()
```

```
  const tr = React.useRef([])
```

```
  const data = [
```

```
    ['JavaScript', 100],
```

```
    ['React', 150],
```

```
    ['React Native', 180],
```

```
    ['Node.js', 200],
```

```
    ['VS Code', 120]
```

```
  ]
```

```
  const onClickButton = (i) => {
```

```
    //การลบแถวออกจากตาราง จะทำให้ลำดับแถวเปลี่ยนไปจากเดิม
```

```
    //ซึ่งอาจไม่ตรงกับลำดับอ้างอิงของแถวนั้นที่ได้กำหนดไว้ล่วงหน้า
```

```
    //ดังนั้น เราต้องตรวจสอบลำดับที่แท้จริงอีกครั้ง เพื่อใช้ในการลบ
```

```
    const index = tr.current[i].rowIndex
```

```
    table.current.deleteRow(.....)
```

```
  }
```

การสร้างค่าอ้างอิง (Refs)

react/app1/src/refs-array.js (ต่อ)

```
const onMouseOverRow = (i) => {
  tr.current[i].style.backgroundColor = .....
}

const ..... = (i) => {
  ..... = 'white'
}

const tableStyles = { margin: 'auto', marginTop: 30 }

return (
  <table ref={.....} border="1" cellPadding={5} style={.....} >
    <tr><th>Product</th><th>Price</th><th>Delete</th></tr>
    {
```

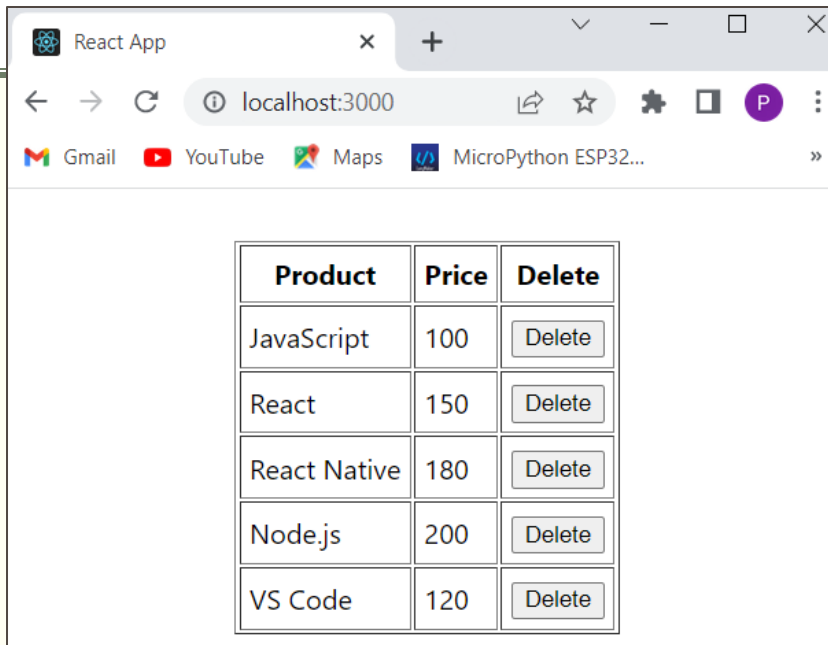
```
data.map((item, i) => {
  return (
    <tr ref={el => tr.current[i] = el}
      onMouseOver={() => onMouseOverRow(i)}
      onMouseOut={.....}
    >
      <td>{item[0]}</td>
      <td>{.....}</td>
      <td><button onClick={() => onClickButton(i)}>
        .....</button></td>
    </tr>
  )
})
}
</table>
)
```


การสร้างค่าอ้างอิง (Refs)

react/app1/src/App.js

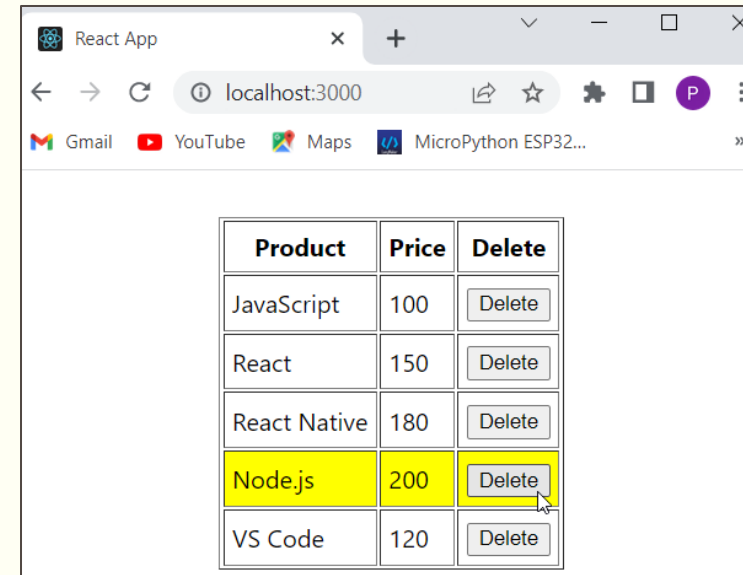
```
import ..... from 'react'
import RefsArray from './refs-array'

export default function App() {
  return .....
}
```



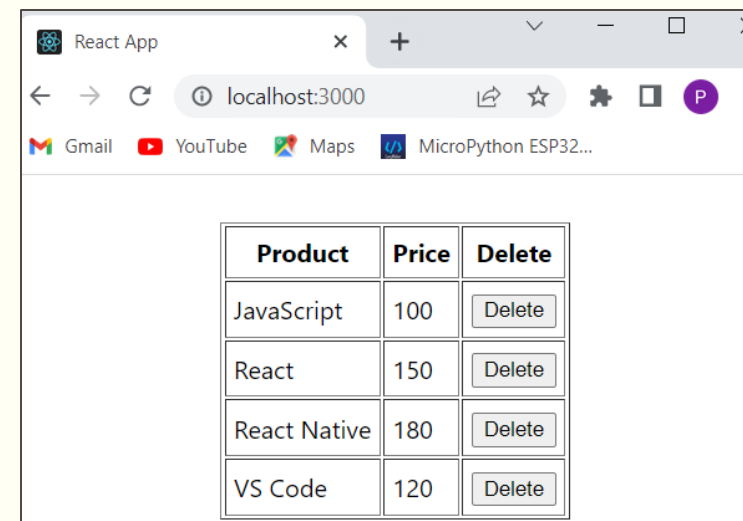
A screenshot of a web browser window titled 'React App' showing a table with three columns: Product, Price, and Delete. The table contains five rows of data. The 'Delete' button in the last row is highlighted.

Product	Price	Delete
JavaScript	100	Delete
React	150	Delete
React Native	180	Delete
Node.js	200	Delete
VS Code	120	Delete



A screenshot of a web browser window titled 'React App' showing a table with three columns: Product, Price, and Delete. The table contains five rows of data. The 'Node.js' row is highlighted in yellow.

Product	Price	Delete
JavaScript	100	Delete
React	150	Delete
React Native	180	Delete
Node.js	200	Delete
VS Code	120	Delete



A screenshot of a web browser window titled 'React App' showing a table with three columns: Product, Price, and Delete. The table contains five rows of data.

Product	Price	Delete
JavaScript	100	Delete
React	150	Delete
React Native	180	Delete
VS Code	120	Delete

ความรู้เพิ่มเติมเกี่ยวกับ Array Destructuring

Array Destructuring เป็นเทคนิคการนำค่าจากอาร์เรย์มากำหนดให้แก่ตัวแปรหลายๆ ตัว เพื่อใช้งานอื่นๆ ต่อไป ซึ่งตามปกตินั้นหากเราต้องการเข้าถึงค่าของสมาชิกแต่ละตัวในอาร์เรย์ ก็อาจต้องใช้เลขลำดับในการอ้างอิง เช่น

```
let arr = [1, 2, 3, 4, 5]
let a = arr[0]
let b = arr[1]
```

แต่ยังมีวิธีการที่ง่ายกว่าหากต้องการนำค่าจากอาร์เรย์ไปกำหนดให้กับตัวแปรแบบแยกกัน นั่นคือใช้วิธีการที่เรียกว่า Array Destructuring ในลักษณะดังนี้

```
let [a, b, c] = [1, 2, 3]
console.log(a)    //1
console.log(b)    //2
console.log(c)    //3
```

ความรู้เพิ่มเติมเกี่ยวกับ Array Destructuring

หรือหากเราต้องการใช้ข้อมูลเพียงบางส่วน ก็อาจสร้างตัวแปรเท่าที่จำเป็น และอาจแทนส่วนที่เหลือด้วย Spread Operator (...) นอกจากนี้ ยังอาจป้องกันกรณีที่มีจำนวนสมาชิกในอาร์เรย์ไม่ครบตามจำนวนตัวแปรด้วยการกำหนดค่าดีฟอลต์แทนค่าที่อาจขาดหายไป เช่น

```
let [a, b] = [1, 2, 3, 4, 5]
```

```
//a = 1, b = 2
```

```
let [c, d, ...e] = [6, 7, 8, 9]      //... คือ spread operator
```

```
//c = 6, d = 7, e = [8, 9], e[0] = 8, e[1] = 9
```

```
let [f, g, h=12, i] = [10, 11]      //กำหนดค่าดีฟอลต์ของ h เป็น 12
```

```
//f = 10, g = 11, h = 12, i = undefined
```

การจัดเก็บข้อมูลสถานะด้วย State

React Hook ที่เราจะศึกษาในลำดับต่อไปคือ State ซึ่งเป็นการจัดเก็บข้อมูลสถานะของคอมโพเนนต์โดยมีลักษณะที่สำคัญคือ เมื่อข้อมูลที่จัดเก็บในแบบ State เปลี่ยนแปลงไป คอมโพเนนต์ที่เป็นเจ้าของข้อมูลนั้นจะแสดงผลใหม่ทันที (re-render) ทั้งนี้ State ถือเป็นสิ่งสำคัญอย่างยิ่งที่เราต้องนำไปใช้งานอยู่ตลอดทั้งใน React และ React Native สำหรับแนวทางการใช้งานนั้น จะแยกพิจารณาระหว่าง Class Component และ Function Component ดังนี้

- **การกำหนด State สำหรับ Class Component**

สำหรับกรณีของ Class Component นั้น เราจะสร้าง State ให้เป็นพร็อพเพอร์ตี้ของคลาส แล้วจัดเก็บข้อมูลย่อยๆ แต่ละอย่างในแบบออบเจกต์ ซึ่งอาจเลือกใช้วิธีใดวิธีหนึ่งดังต่อไปนี้

- วิธีที่ 1 สร้าง State ในคอนสตรัคเตอร์ เช่น

```
class Cart extends React.Component {  
  constructor() {  
    super()  
    this.state = { numItems: 0 }  
  }  
  ...  
}
```

การจัดเก็บข้อมูลสถานะด้วย State

- วิธีที่ 2 สร้าง State ในแบบพร็อพเพอร์ตี้ของคลาสโดยตรง เช่น

```
class Cart extends React.Component {  
  state = { numItems: 0 }  
  ...  
}
```

ภายในคอมโพเนนต์ เราสามารถนำค่าจาก State ไปใช้งานตามต้องการ สำหรับกรณีของ Class Component ให้อ้างถึงค่าในรูปแบบดังนี้

```
this.state.<name>
```

โดย name ก็ชื่อหรือพร็อพเพอร์ตี้ของข้อมูลที่จัดเก็บในออบเจกต์ State ที่เราต้องการอ่านค่านั่นเอง เช่น

```
let n = this.state.numItems
```

ส่วนการแก้ไขค่าของ State จะต้องใช้เมธอดที่ชื่อ setState() โดยเรียกเมธอดนี้เมื่อต้องการเปลี่ยนค่า State แล้วกำหนดค่าใหม่ในแบบออบเจกต์ เช่น

```
this.setState({numItems: 2})
```

การจัดเก็บข้อมูลสถานะด้วย State

react/app1/src/state-class.js

```
import React from 'react'
```

```
export default class Cart extends React.Component {  
  state = { numItems: 0 }
```

```
  onClickAddCart = () => {  
    let n = this.state.numItems  
    n++  
    this.setState({ numItems: n })  
  }
```

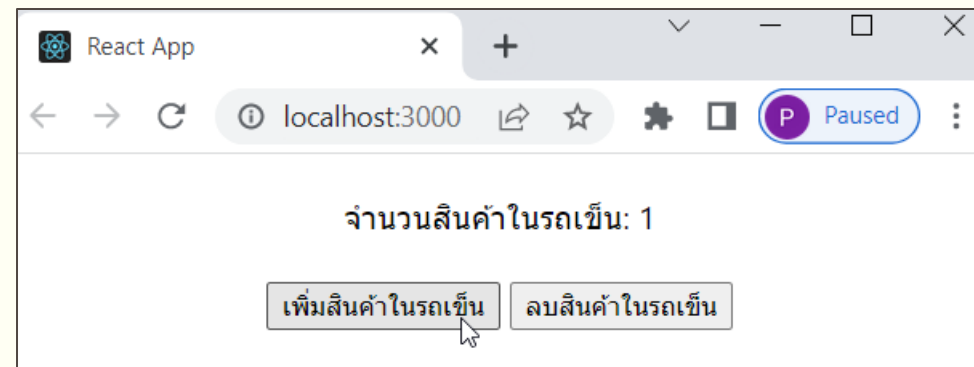
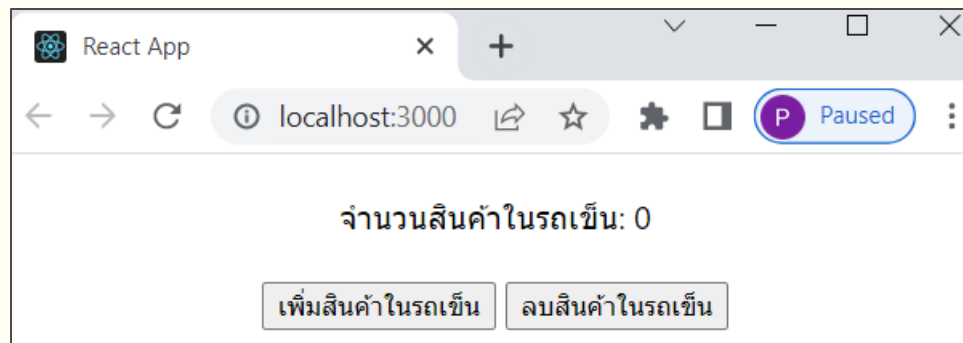
```
  onClickDeleteCart = () => {  
    if (this.state.numItems > 0) {  
      this.setState({ numItems: this.state.numItems - 1 })  
    }  
  }  
  
  render() {  
    return (  
      <div style={{textAlign:'center', marginTop: 20}}>  
        <div>จำนวนสินค้าในรถเข็น: {this.state.numItems}</div><br/>  
        <button onClick={this.onClickAddCart}>เพิ่มสินค้าในรถเข็น</button>&nbsp;<br/>  
        <button onClick={this.onClickDeleteCart}>ลบสินค้าในรถเข็น</button>  
      </div>  
    )  
  }  
}
```

การจัดเก็บข้อมูลสถานะด้วย State

react/app1/src/App.js

```
import React from 'react'
import Cart from './state-class'

export default function App() {
  return <Cart/>
}
```



การจัดเก็บข้อมูลสถานะด้วย State

- **การกำหนด State สำหรับ Function Component**

การใช้ State ร่วมกับ Function Component จะมีขั้นตอนบางส่วนที่แตกต่างไปจาก Class Component ซึ่งสามารถสรุปหลักการที่สำคัญได้ดังนี้

- การสร้าง State สำหรับ Function Component มีฟังก์ชันในกลุ่ม React Hook ให้ใช้งานโดยตรงอยู่แล้ว นั่นคือ `useState()` ซึ่งอาจนำเข้าฟังก์ชันนี้โดยตรงจากโมดูล 'react' หรือเรียกผ่านคอมโพเนนต์ React ก็ได้

//วิธีที่ 1

```
import React, {useState} from 'react'

function MyComponent() {
  let [...] = useState()
}
```

//วิธีที่ 2

```
import React from 'react'

function MyComponent() {
  let [...] = React.useState()
}
```


การจัดเก็บข้อมูลสถานะด้วย State

- ภายในคอมโพเนนต์ เราต้องกำหนดข้อมูลของ State ด้วยวิธีการแบบ Array Destructuring พร้อมกำหนดค่าเริ่มต้นให้กับมันด้วยฟังก์ชัน useState() เช่น สมมติว่าเราต้องการสร้าง State เพื่อเก็บขนาดของฟอนต์ภายใน Function Component ก็อาจกำหนดค่า ดังนี้

```
import React, {useState} from 'react'

function MyComponent() {
  let [fontSize, setFontSize] = useState(16)
  ...
}
```

- ◎ [fontSize, setFontSize] คือการสร้างตัวแปรแบบ Array Destructuring
- ◎ fontSize คือตัวแปรที่ใช้เก็บค่า State
- ◎ setFontSize คือชื่อฟังก์ชันที่จะใช้ในการเปลี่ยนค่า State ซึ่งส่วนใหญ่เรานิยมให้ขึ้นต้นด้วยคำว่า set แล้วตามด้วยชื่อตัวแปรแต่ไม่ใช่ข้อบังคับ
- ◎ useState(16) เป็นฟังก์ชันที่ใช้สร้างและกำหนดค่าเริ่มต้นให้แก่ State โดยในที่นี้คือตัวแปร fontSize ซึ่งจะมีค่าเริ่มแรกเป็น 16 หรือ fontSize = 16 นั่นเอง

การจัดเก็บข้อมูลสถานะด้วย State

- การนำค่าของ State ไปใช้งาน ก็ระบุชื่อตัวแปรนั้นโดยตรงเหมือนกับตัวแปรทั่วไป
- สำหรับการแก้ไขหรืออัปเดตค่า State ให้กำหนดผ่านทางฟังก์ชันที่ระบุไว้ใน Array Destructuring ตอนสร้าง State ซึ่งจากโค้ดที่ผ่านมาคือ `setFontSize` แต่หากเราแก้ไขค่าของตัวแปร State โดยตรงจะเกิดข้อผิดพลาด เช่น

```
function MyComponent() {  
  let [fontSize, setFontSize] = useState(16)  
  ...  
  //สมมติว่าเมื่อคลิกปุ่ม zoom แล้วให้เพิ่มขนาดฟอนต์  
  const onClickButtonZoomIn = () => {  
    let newSize = fontSize + 2 //อ้างอิงตัวแปรใน State  
    setFontSize(newSize)      //อัปเดตค่าของ State
```

```
    /*  
    fontSize = 18 //Error เพราะแก้ไขค่าที่ตัวแปร State โดยตรง  
    fontSize = fontSize + 2 //Error  
    setFontSize(++fontSize) //Error (ผลลัพธ์ของ ++ จะเก็บไว้ในตัว  
                             แปรนั้น)  
    */  
  }  
  ...  
  <button onClick={onClickButtonZoomIn}>Zoom In</button>  
}
```

// หรือแบบง่ายๆ คือ เรียกฟังก์ชันที่ใช้อัปเดตค่าของ State โดยตรง เช่น

```
<button onClick={() => setFontSize(fontSize + 2)}>Zoom In</button>
```

การจัดเก็บข้อมูลสถานะด้วย State

- หลังจากที่เราเปลี่ยนค่าของ State คอมโพเนนต์จะแสดงผลใหม่หรือ re-render ทันที เช่นเดียวกับการเปลี่ยน State ใน Class Component ดังที่กล่าวมาแล้ว ทั้งนี้ เราสามารถเพิ่มการจัดเก็บข้อมูลอื่นๆ ไว้ใน State ได้ตามต้องการ เช่น

```
let [fontSize, setFontSize] = useState(16)
let [color, setColor] = useState()      //ไม่ระบุค่าเริ่มต้นก็ได้
let [fontWeight, setFontWeight] = useState('normal')
let [disabled, setDisabled] = useState(false)
```

การจัดเก็บข้อมูลสถานะด้วย State

react/app1/src/state-func.js

```
import React, {.....} from 'react'

export default function ..... {
  let [text, setText] = useState('Hello World')
  let [size, .....] = React.useState(16)

  const onClickSetText = () => {
    let t = prompt('กำหนดข้อความ')
    if (t) {
      setText(t)
    }
  }

  const onClickZoomIn = () => {
    let newSize = size + 1
    setSize(.....)
  }
```

```
let msgboxSyle = {
  display: 'inline-block',
  width: 350,
  fontSize: size,    //ใช้ค่าจาก State
  backgroundColor: '#ccc',
  padding: 5,
  textAlign: 'left'
}

return (
  <div style={{textAlign:'center', marginTop:20}}>
    <div style={.....}>{text}</div><br/><br/>
    <button onClick={.....}>ข้อความ</button>&nbsp;
    <button onClick={onClickZoomIn}>.....</button>&nbsp;
    <button onClick={() => setSize(size - 1)}>.....</button>
  </div>
)
```

การจัดเก็บข้อมูลสถานะด้วย State

react/app1/src/App.js

```
import React from 'react'
import MessageBox from .....

export default function App() {
  return .....
}
```

